

实验

这份文件和[实验指导](#)的区别：**实验指导讲“为什么”，这份讲“怎么做”**。照着往下敲就行。

每步都写了三件事：**敲什么** → **该看到什么** → **要记下什么**。下面所有输出都是真跑出来的，你看到的应该一模一样。

开始之前

1. 装依赖（只这一步需要联网）

```
pip install cryptography
```

装不上就换国内镜像：`pip install -i https://pypi.tuna.tsinghua.edu.cn/simple cryptography`

2. 进到 code 目录

相对路径

```
cd aes-lab/code
```

windows下的绝对路径,linux下直接cd 即可

```
cd /d d:\\aes-lab\\code
```

后面所有命令都在这个目录里敲。**不要**跑到别的目录去。

3. 认识你要打交道的文件

文件	它是什么
alice_encrypt.py	Alice 的加密程序（有密钥）
bob_decrypt.py	Bob 的解密程序（有密钥）
task1 ~ task6	你要完成的六个任务
task4_forge.py	你的主交付物 ：密文伪造工具（没有密钥）

第 0 步：环境自检（必做，1 分钟）

敲：

```
python check_env.py
```

该看到：

FIPS-197 附录 C.1 已知答案测试

```
密钥      : 000102030405060708090a0b0c0d0e0f
明文      : 00112233445566778899aabbccddeeff
期望密文  : 69c4e0d86a7b0430d8cdb78070b4c55a
实际密文  : 69c4e0d86a7b0430d8cdb78070b4c55a  通过
解密回明文: 00112233445566778899aabbccddeeff  通过
```

PKCS#7 填充 : 通过

```
=====
环境自检全部通过，可以开始实验。
```

要记下：这里两个 通过。

⚠ 如果这一步就不过，后面全是白做。先解决环境问题，见指导书附录 A。

你看到的 cryptography: 后面是版本号（如 50.0.1）。如果你看到 0.0.0-devshim，说明这台机器在用实验包自带的接口垫片——**计算结果完全一样**（AES-ECB 是确定性的），不影响任何结论。

第 1 步：建立"历史密文库"（必做）

敲：

```
python alice_encrypt.py --build-history
```

该看到：

```
已生成历史密文库: ...\\data\\history.json
格式: aligned
条数: 6
```

文件里只有密文和公开的字段标注，**没有密钥**。

要记下： 这 6 条历史消息是整场攻击的"素材库"。

打开 data/history.json 看一眼 —— 里面**只有密文**和明文字段标注，没有密钥。攻击者 Mallory 拿到的就是这个文件。

任务一：看清密文的分组结构

敲：

```
python task1_basics.py
```

该看到（节选）：

明文：'From: Alice\nTo: Bob\nDate: 2026-03-15\nPlace: Library'
明文长度：51 字节

密文（hex）：b9b0d45b128f5e3076662263a382b46b02cc8e3516373b...

密文字节数 ： 64

分组数 ： 4

第 0 个分组：[0:16] b9b0d45b128f5e3076662263a382b46b |...[...^0vf
第 1 个分组：[16:32] 02cc8e3516373b4c02d4bc0fcae1f5 |...5·7;L·
第 2 个分组：[32:48] 4a8d97f6dabca66a486c1bbe82b47424 |J.....jH.
第 3 个分组：[48:64] 498adca0cc465fed93785cd5d22282d4 |I.....F_...)

要记下这三件事（脚本最后会再提醒你一遍）：

1. 密文 = 16 字节一组的方块，可以直接切片
2. 分组之间彼此独立，换位置也能解密
3. 变密钥 → 整条密文都变；换分组 → 只有局部变。**后者就是攻击入口**

这一节的思考题： 明文 51 字节，密文 64 字节，多出来的 13 字节是什么？（答案在脚本里写了：PKCS#7 填充）

任务二：发现 ECB 的模式泄露

敲：

```
python task2_pattern.py
```

该看到（节选）：

【分组0 (From)】不同的密文分组共 3 种	出现的消息	共同
密文分组		

08096a96a3a0475152d184e5931b4b1a	M1, M2, M3, M6	Alice
【分组3 (Place)】不同的密文分组共 4 种		
050d523d65fc1ba9c33a5f9cc695f9ea	M1, M3, M6	Library

要记下:

- M1 / M2 / M3 / M6 的**分组 0 完全相同** → 同一个人发的
- M1 / M2 / M4 / M6 的**分组 2 完全相同** → 同一天
- M1 / M3 / M6 的**分组 3 完全相同** → 同一个地点
- M1 和 M2 **只差最后一个分组** → 这两条只有地点不一样

这一步是实验的关键转折点。 Mallory 手上没有密钥，但他已经知道消息的 **结构**了——这叫**模式泄露**。密钥依然安全，AES 没有被破解。

任务三：第一次搬运分组（会先失败一次）

敲:

```
python task3_splice.py
```

该看到（节选）：

先是一个**失败**的例子：

```
尝试 1: 把 M2 的最后一个分组接到 M1 上
Bob 看到:
From: Alice
To: Bob
Date: 2026-03-15
Place: Libr3-15      <- 变成了乱码
Place: Gym           <- 还多出一行
```

然后是一个**成功**的例子：

尝试 2: 换成 M3 的分组 2 (换日期)

Bob 看到:

From: Alice

To: Bob

Date: 2026-03-22 <- 日期真的被改掉了

Place: Library

要记下: 为什么第一次会失败?

因为格式 A (自然写法) 下, **分组边界切在了字段中间**。M1 的第 3 分组装的是 ary 加填充, M2 的最后分组装的是 3-15\nPlace: Gym 加填充 —— 两者根本不是同一个字段, 拼起来自然是乱的。

🔥 **这是全实验最重要的一课。** "搬运分组"要成功, 前提是**分组边界要恰好落在字段边界上**。下一节你会看到, 程序员为了让解析简单而做的"定长对齐", 恰好就把这件事给办成了。

任务四【主交付物】: 写你的密文伪造工具

先跑一次官方版看看长什么样:

```
python task4_forge.py
```

它会问你三个问题, **输入方式很宽松**: 输 M1 或序号 1 都行, 字段名大小写也不敏感。

[1/3] 要伪造哪一条消息? (输入编号如 M1, 或序号 1; 回车默认 M1)

> M1

[2/3] 要改哪个字段? 可选 From(发件人) / To(收件人) / Date(日期) / Place(地点)
(回车默认 Place)

> Place

[3/3] 从哪一条消息里取「地点」这个分组?

编号	该字段的内容	分组是否与目标不同
M1	Library	相同 (换了等于没换)
M2	Gym	不同
M3	Library	相同 (换了等于没换)
M4	Office	不同
M5	Cafe	不同
M6	Library	相同 (换了等于没换)

> M2

该看到：

伪造完成

目标消息 M1: Alice -> Bob 2026-03-15 Library

取块来源 M2: Alice -> Bob 2026-03-15 Gym

取块配方：

分组0:	08096a96a3a0475152d184e5931b4b1a	来自 M1.分组0
分组1:	43f16df7956aa0ba8a53ece25932f60b	来自 M1.分组1
分组2:	90cd57ec0e9013c73633feca93e40681	来自 M1.分组2
分组3:	afed40e112212decde60a35b5fc8422e	来自 M2.分组3 <-- 换掉了

已写入: ...\\data\\forged.ct

然后关掉它，假装你是 Bob，去解密这个文件：

```
python bob_decrypt.py -i ../data/forged.ct
```

该看到：

解密结果：

```
-----  
From: Alice  
To: Bob  
Date: 2026-03-15  
Place: Gym  
-----
```

⚠ Bob 无法判断这条密文是不是原装的 — ECB 没有任何完整性保护。

要记下： Place: Gym —— 你成功了。Alice 从来没发过这条消息。

你的交付物

现在**自己写**一个伪造工具。要求：

1. 只能 import block_utils, **不能** import ecb_common (ecb_common 里有密钥, Mallory 是拿不到的)
2. 能指定：改哪条消息、改哪个字段、从哪条消息取块
3. 输出伪造后的密文文件

自检方法（这条能证明你没偷看密钥）：

```
python -c "import ast; t=ast.parse(open('task4_forge.py',encoding='utf-8').read())"
```

输出应该是 ['__future__', 'block_utils', 'pathlib'] —— 没有 ecb_common。

任务五【挑战】：拼一条从未存在过的消息

敲：

```
python task5_challenge.py
```

该看到（节选）：

伪造密文：8a75dc86c8b472ad42785f678d721fdb43f16df7956aa0ba...

Bob 解密后看到：

From: Carol

To: Bob

Date: 2026-03-22

Place: Gym

字段溯源：

From	= Carol	来自 M4
To	= Bob	来自 M1
Date	= 2026-03-22	来自 M3
Place	= Gym	来自 M2

这条消息**没有任何人发送过** — 它是四个分组的拼装件，但格式完整、语义自洽、解密不报错。

要记下：四个分组来自**四条不同的消息**，拼出来却完全通顺。"Carol 在 3 月 22 日通知 Bob 去 Gym" 这件事，历史上从未发生。

你的挑战：自己设计至少两条新的拼装配方，要求分组来自 ≥ 3 条不同消息，写进报告。

任务六：为什么换成 CBC / GCM 就防住了

敲：

```
python task6_defense.py
```

该看到（节选）：

CBC: 被拒绝 -> ValueError: 填充字节非法: 0x6f (PKCS#7)
GCM: 被拒绝 -> InvalidTag: 认证标签校验失败

要记下这里两个反直觉的结论（报告里要写）：

- 1. **CBC 并不真的“防住”了分组搬运。** 它加的是**链接**，不是**完整性**。它只是让“前驱对不上”的情况失败而已——如果两条消息前缀相同，搬运**依然成功**（任务三已经见过）。
- 2. **GCM 里真正拦住攻击的是认证标签，不是分组结构。** 分组照样能搬，但标签会对不上，解密直接抛 InvalidTag。

结论：“用了 AEAD 就安全”的重点在**认证**，不在分组怎么排。

收尾：检查与交作业

一键确认全部 11 个步骤都通过：

```
python run_all.py
```

该看到：

全部 11 个步骤通过

交：

- 1. 你的伪造程序（任务四）
- 2. 实验报告（格式见指导书 [第 5 节](#)）

卡住了怎么办

你遇到	原因	怎么办
ModuleNotFoundError: No module named 'cryptography'	没装	pip install cryptography
找不到 data/history.json	跳过第 1 步了	python alice_encrypt.py --build-history
ValueError: 填充字节非法	密文被改动过	这不是 bug —— 正是任务六要观察的现象

ValueError: 数据长度不是分组长度的整数倍	忘了 PKCS#7 填充	ECB/CBC 不会 自动填充
Windows 上重定向到文件后中文乱码	cmd.exe 是 GBK 编码	计算结果没错 , 先 chcp 65001, 详见指导书附录 A.4
文档里的数字和我跑的不一样	你改了密钥或消息格式	检查是否与 ecb_common.py 一致; 攻击是否依然成立才是重点

完整排错见[指导书附录 A](#)。